

PROGRAMA DE LA ASIGNATURA

DEVOPS

Integración, Entrega y Operaciones Continuas

Programa orientado al desarrollo de competencias profesionales para el rol de DevOps Engineer

Nivel sugerido	Universitario / Técnico Superior
Duración	Un cuatrimestre (16 semanas)
Carga horaria	96 horas: 32 teóricas y 64 prácticas
Modalidad	Teórico-práctica con laboratorio
Orientación	Formación profesional DevOps

Documento académico de referencia

Datos generales y supuestos de diseño

- Nivel: universitario, técnico superior o formación profesional avanzada.
- Duración propuesta: un cuatrimestre de 16 semanas.
- Carga horaria sugerida: 96 horas.
- Distribución: 32 horas teóricas y 64 horas prácticas.
- Modalidad: teórico-práctica con laboratorio.
- Conocimientos previos recomendados: programación básica, sistemas operativos, redes, bases de datos y control de versiones.

Contenido del programa

5. Fundamentación
6. Objetivos
7. Competencias y resultados de aprendizaje
8. Programa sintético
9. Programa analítico
10. Metodología de enseñanza
11. Actividades prácticas
12. Proyecto integrador
13. Evaluación y condiciones de aprobación
14. Cronograma propuesto
15. Recursos tecnológicos sugeridos
16. Bibliografía
17. Vigencia y revisión del programa

5. Fundamentación

La evolución de los sistemas informáticos, los servicios en la nube y las aplicaciones distribuidas ha generado la necesidad de integrar de forma continua las actividades de desarrollo de software, infraestructura tecnológica, seguridad y operaciones.

En los modelos tradicionales, los equipos de desarrollo y operaciones trabajan frecuentemente de manera separada. Esta división puede producir demoras en las entregas, dificultades para reproducir entornos, errores de configuración, fallas durante los despliegues y problemas de comunicación entre los distintos equipos.

La cultura DevOps propone una forma de trabajo colaborativa basada en la automatización, la integración continua, la entrega continua, la infraestructura como código, la observabilidad y la mejora permanente de los procesos.

La asignatura busca que el estudiante comprenda el ciclo de vida completo de una aplicación, desde la escritura y administración del código fuente hasta su construcción, prueba, publicación, despliegue, monitoreo y mantenimiento en ambientes productivos.

El enfoque estará orientado a la resolución de situaciones reales mediante laboratorios y proyectos en los que se integrarán herramientas de control de versiones, automatización, contenedores, orquestación, infraestructura como código, plataformas en la nube, monitoreo y seguridad.

La asignatura no se limitará al aprendizaje aislado de herramientas. Se priorizará la comprensión de los principios que permiten diseñar procesos confiables, repetibles, seguros, escalables y adaptables a diferentes tecnologías.

6. Objetivos

6.1. Objetivo general

Desarrollar en el estudiante las competencias necesarias para diseñar, automatizar, implementar, monitorear y mantener procesos de integración, entrega y operación continua de sistemas informáticos, aplicando principios y prácticas propias de la cultura DevOps.

6.2. Objetivos específicos

1. Comprender los fundamentos culturales, organizacionales y técnicos de DevOps.
2. Analizar el ciclo de vida del desarrollo de software e identificar oportunidades de automatización.
3. Administrar repositorios de código mediante sistemas de control de versiones.
4. Diseñar flujos de trabajo colaborativos utilizando ramas, solicitudes de incorporación de cambios y revisiones de código.
5. Automatizar tareas operativas mediante scripts.
6. Crear procesos de integración continua para compilar, validar y probar aplicaciones.
7. Diseñar procesos de entrega y despliegue continuo.
8. Construir, administrar y publicar imágenes de contenedores.
9. Implementar aplicaciones contenerizadas en entornos locales y distribuidos.
10. Administrar configuraciones, variables de entorno y secretos.
11. Definir infraestructura mediante archivos de configuración versionables.
12. Aplicar técnicas básicas de monitoreo, registro de eventos, generación de métricas y trazabilidad.
13. Establecer indicadores relacionados con la disponibilidad, el rendimiento y la confiabilidad de los servicios.
14. Incorporar controles de seguridad dentro del ciclo de desarrollo y despliegue.
15. Participar en la gestión de incidentes y en el análisis de fallas.

16. Integrar los conocimientos adquiridos en un proyecto DevOps completo.

7. Competencias y resultados de aprendizaje

7.1. Competencias técnicas

El estudiante desarrollará capacidades para:

- Utilizar sistemas Linux desde la línea de comandos.
- Configurar redes y servicios básicos.
- Administrar repositorios Git.
- Automatizar tareas mediante Bash, Python u otro lenguaje de scripting.
- Crear pipelines de integración y entrega continua.
- Construir imágenes de contenedores.
- Implementar aplicaciones mediante Docker y plataformas de orquestación.
- Definir infraestructura como código.
- Configurar herramientas de monitoreo y observabilidad.
- Gestionar registros, métricas, alertas y trazas.
- Aplicar controles básicos de seguridad.
- Documentar procesos técnicos y operativos.

7.2. Competencias profesionales

El estudiante deberá demostrar capacidad para:

- Trabajar de manera colaborativa.
- Comunicar problemas técnicos.
- Documentar decisiones y procedimientos.
- Analizar riesgos operativos.
- Resolver incidentes de manera metódica.
- Evaluar alternativas tecnológicas.
- Incorporar la mejora continua.
- Asumir responsabilidad sobre el funcionamiento integral de los servicios.

7.3. Resultados de aprendizaje

Al aprobar la materia, el estudiante podrá:

- Diseñar un flujo DevOps desde el repositorio hasta producción.
- Construir una aplicación mediante un pipeline automatizado.
- Ejecutar pruebas dentro del proceso de integración continua.
- Empaquetar una aplicación en contenedores.
- Desplegar servicios en un entorno de orquestación.
- Crear infraestructura reproducible mediante código.
- Implementar tableros de monitoreo y reglas de alerta.
- Aplicar prácticas básicas de DevSecOps.
- Elaborar documentación técnica y procedimientos de recuperación.
- Presentar y defender un proyecto integrador funcional.

8. Programa sintético

Unidad I: Fundamentos de DevOps: Cultura DevOps. Ciclo de vida del software. Colaboración entre desarrollo, operaciones y seguridad.

Unidad II: Sistemas operativos, redes y automatización: Linux. Línea de comandos. Procesos. Servicios. Redes. Protocolos. Automatización mediante scripts.

Unidad III: Control de versiones y trabajo colaborativo: Git. Repositorios. Ramas. Etiquetas. Conflictos. Revisión de código. Estrategias de trabajo colaborativo.

Unidad IV: Integración continua: Construcción automatizada. Pruebas. Calidad de código. Artefactos. Pipelines de integración.

Unidad V: Entrega y despliegue continuo: Automatización de entregas. Ambientes. Versionado. Estrategias de despliegue. Reversión de versiones.

Unidad VI: Contenedores: Imágenes. Contenedores. Registros. Redes. Volúmenes. Aplicaciones multicontenedor.

Unidad VII: Orquestación de contenedores: Arquitectura de Kubernetes. Objetos principales. Despliegues. Servicios. Configuración. Escalabilidad.

Unidad VIII: Infraestructura como código: Automatización de infraestructura. Gestión de configuración. Estado. Módulos. Aprovisionamiento.

Unidad IX: Observabilidad, confiabilidad e incidentes: Registros, métricas y trazas. Monitoreo. Alertas. Indicadores de servicio. Gestión de incidentes.

Unidad X: DevSecOps y proyecto integrador: Seguridad dentro del pipeline. Gestión de secretos. Análisis de dependencias. Proyecto final DevOps.

9. Programa analítico

Unidad I: Fundamentos de DevOps

Concepto de DevOps. Antecedentes y evolución. Relación entre desarrollo, operaciones, calidad y seguridad. Objetivos de la cultura DevOps. Colaboración, comunicación, automatización y mejora continua.

Ciclo de vida del desarrollo de software. Planificación, codificación, construcción, prueba, publicación, despliegue, operación y monitoreo.

Metodologías ágiles y DevOps. Retroalimentación continua. Gestión del flujo de trabajo. Reducción de tareas manuales. Automatización de procesos repetitivos.

Responsabilidad compartida sobre los productos y servicios. Equipos interdisciplinarios. Cultura sin culpabilización. Aprendizaje a partir de errores.

Introducción a las prácticas de integración continua, entrega continua, infraestructura como código, observabilidad y DevSecOps.

Unidad II: Sistemas operativos, redes y automatización

Fundamentos de Linux. Estructura del sistema de archivos. Usuarios, grupos y permisos. Administración básica de archivos y directorios.

Procesos y servicios. Variables de entorno. Gestión de paquetes. Registros del sistema. Administración remota mediante SSH.

Introducción a redes. Direccionamiento IP. Puertos. Protocolos TCP y UDP. DNS. HTTP y HTTPS. Resolución de nombres. Conectividad entre servicios.

Uso de herramientas de diagnóstico de red. Pruebas de conectividad. Consulta de puertos y servicios. Análisis básico de errores.

Automatización mediante scripts. Variables, estructuras condicionales, ciclos y funciones. Procesamiento de archivos. Ejecución programada de tareas.

Principios de idempotencia, repetibilidad y control de errores.

Unidad III: Control de versiones y trabajo colaborativo

Concepto de control de versiones. Repositorios locales y remotos. Configuración de Git.

Creación y clonación de repositorios. Área de trabajo, área de preparación e historial. Confirmaciones de cambios.

Ramas y fusiones. Resolución de conflictos. Etiquetas y versiones. Reversión y recuperación de cambios.

Repositorios remotos. Autenticación. Solicitudes de incorporación de cambios. Revisión de código.

Estrategias de ramas. Desarrollo basado en ramas de funcionalidades. Desarrollo basado en una rama principal. Ventajas y limitaciones de cada estrategia.

Archivos ignorados. Manejo responsable de datos sensibles. Documentación mediante archivos README.

Buenas prácticas para mensajes de confirmación y organización del historial.

Unidad IV: Integración continua

Concepto de integración continua. Beneficios y limitaciones. Automatización de la construcción y validación del software.

Estructura de un pipeline. Etapas, trabajos, ejecutores, disparadores y dependencias.

Compilación de aplicaciones. Instalación de dependencias. Ejecución de pruebas unitarias y de integración.

Análisis estático de código. Validación de formato. Cobertura de pruebas. Criterios de aprobación.

Administración de artefactos. Versionado. Almacenamiento y publicación.

Variables del pipeline. Credenciales. Secretos. Ejecución condicional. Matrices de construcción.

Notificaciones y resultados de ejecución. Diagnóstico de fallas en pipelines.

Unidad V: Entrega y despliegue continuo

Diferencias entre integración continua, entrega continua y despliegue continuo.

Ambientes de desarrollo, prueba, preproducción y producción. Promoción de artefactos entre ambientes.

Configuración externa de aplicaciones. Variables de entorno. Separación entre código y configuración.

Versionado semántico. Publicación de versiones. Notas de lanzamiento.

Validaciones posteriores al despliegue. Pruebas de humo. Verificación del estado del servicio.

Reversión automatizada. Rollback y roll-forward. Gestión de cambios y aprobaciones.

- Despliegue recreado.
- Despliegue gradual.
- Rolling update.
- Blue-green.
- Canary.
- Pruebas A/B.

Unidad VI: Contenedores

Concepto de virtualización y contenerización. Diferencias entre máquinas virtuales y contenedores.

Arquitectura de un motor de contenedores. Imágenes, capas y contenedores.

Construcción de imágenes. Archivos de definición. Contexto de construcción. Instrucciones principales.

Buenas prácticas para imágenes pequeñas, reproducibles y seguras.

Ejecución de contenedores. Publicación de puertos. Variables de entorno. Redes y volúmenes.

Persistencia de datos. Montaje de archivos y directorios.

Registros de imágenes. Etiquetado. Publicación y descarga.

Aplicaciones multicontenedor. Definición declarativa de servicios. Dependencias y redes internas.

Diagnóstico de problemas. Registros, inspección y acceso a contenedores.

Unidad VII: Orquestación de contenedores

Necesidad de la orquestación. Arquitectura general de Kubernetes.

Clústeres. Nodos de control y nodos de trabajo. API y componentes principales.

Archivos de manifiesto. Sintaxis YAML. Aplicación declarativa de configuraciones.

Escalamiento horizontal. Actualizaciones progresivas. Recuperación automática.

Sondas de vida, disponibilidad e inicio.

Administración de recursos. Solicitudes y límites de CPU y memoria.

Introducción a la administración de paquetes y plantillas para Kubernetes.

Diagnóstico mediante eventos, registros y descripción de recursos.

- Pods.

- Deployments.
- ReplicaSets.
- Services.
- ConfigMaps.
- Secrets.
- Namespaces.
- Persistent Volumes.
- Persistent Volume Claims.
- Ingress.

Unidad VIII: Infraestructura como código

Concepto de infraestructura como código. Diferencias entre procedimientos manuales y configuraciones declarativas.

Principios de reproducibilidad, versionado, idempotencia y automatización.

Proveedores, recursos, variables, salidas y dependencias.

Planificación y aplicación de cambios. Administración del archivo de estado.

Estado local y remoto. Bloqueo del estado. Riesgos asociados a la pérdida o modificación indebida.

Módulos reutilizables. Organización de proyectos.

Entornos y espacios de trabajo. Separación de configuraciones.

Gestión de configuración de servidores. Inventarios, tareas, roles y plantillas.

Aprovisionamiento de recursos en entornos locales o plataformas en la nube.

Validación, revisión y ejecución de infraestructura mediante pipelines.

Unidad IX: Observabilidad, confiabilidad e incidentes

Diferencias entre monitoreo y observabilidad.

Recolección y centralización de registros. Estructuración de eventos. Niveles de registro.

Métricas de infraestructura y aplicaciones. Contadores, indicadores, histogramas y resúmenes.

Tableros de visualización. Consultas. Representación de tendencias.

Alertas. Umbrales. Reducción de falsos positivos. Notificaciones.

Indicadores de nivel de servicio. Objetivos de nivel de servicio. Acuerdos de nivel de servicio.

Disponibilidad, latencia, tasa de errores y consumo de recursos.

Introducción a la ingeniería de confiabilidad de sitios. Presupuesto de errores.

Gestión de incidentes. Identificación, clasificación, contención, resolución y recuperación.

Análisis posterior al incidente. Elaboración de informes sin culpabilización. Acciones correctivas y preventivas.

Copias de seguridad. Recuperación ante fallas. Continuidad operativa.

- Registros.
- Métricas.
- Trazas.

Unidad X: DevSecOps y proyecto integrador

Integración de la seguridad dentro del ciclo de vida del software.

Principio de desplazamiento de la seguridad hacia etapas tempranas.

Gestión de identidades y accesos. Principio de mínimo privilegio.
 Gestión de secretos. Credenciales, tokens, certificados y claves.
 Análisis de dependencias. Identificación de componentes vulnerables.
 Análisis estático y dinámico. Escaneo de imágenes de contenedores.
 Seguridad de pipelines. Protección de ramas. Firmado y validación de artefactos.
 Seguridad de la cadena de suministro de software.
 Políticas como código. Validaciones automáticas.
 Diseño e implementación del proyecto integrador.

10. Metodología de enseñanza

La asignatura se desarrollará mediante una modalidad teórico-práctica, con predominio de actividades de laboratorio.

Cada tema comenzará con una explicación conceptual y continuará con una demostración técnica. Posteriormente, los estudiantes resolverán actividades individuales o grupales en ambientes controlados.

Se promoverá que los estudiantes documenten los procedimientos realizados, los problemas encontrados, las decisiones adoptadas y las soluciones implementadas.

Las herramientas concretas podrán variar, siempre que permitan alcanzar los resultados de aprendizaje establecidos.

Se aplicarán las siguientes estrategias:

- Clases conceptuales.
- Demostraciones prácticas.
- Laboratorios guiados.
- Resolución de problemas.
- Análisis de errores reales.
- Trabajo colaborativo con repositorios.
- Estudio de casos.
- Simulación de incidentes.
- Elaboración de documentación técnica.
- Desarrollo progresivo de un proyecto integrador.

11. Actividades prácticas

Trabajo práctico 1: Administración básica de Linux: Creación de usuarios, configuración de permisos, administración de procesos y automatización de tareas mediante scripts.

Trabajo práctico 2: Repositorio colaborativo: Creación de un repositorio, uso de ramas, resolución de conflictos y revisión de código.

Trabajo práctico 3: Pipeline de integración continua: Construcción de una aplicación, instalación de dependencias, ejecución de pruebas y generación de un artefacto.

Trabajo práctico 4: Contenerización: Creación de una imagen de contenedor para una aplicación y publicación en un registro.

Trabajo práctico 5: Aplicación multicontenedor: Implementación de una aplicación compuesta por servicio web, API y base de datos.

Trabajo práctico 6: Despliegue orquestado: Despliegue de la aplicación mediante Kubernetes, incluyendo servicios, configuraciones y almacenamiento.

Trabajo práctico 7: Infraestructura como código: Definición automatizada de infraestructura y ejecución controlada de cambios.

Trabajo práctico 8: Observabilidad: Configuración de métricas, registros, tableros y alertas.

Trabajo práctico 9: Seguridad del pipeline: Análisis de dependencias, secretos e imágenes de contenedores.

Trabajo práctico 10: Simulación de incidente: Detección de una falla, análisis de causa, recuperación del servicio y elaboración de un informe posterior.

12. Proyecto integrador

El proyecto integrador consistirá en implementar el ciclo DevOps completo de una aplicación.

El trabajo deberá incluir como mínimo:

1. Repositorio de código correctamente organizado.
2. Estrategia de ramas y revisión de cambios.
3. Aplicación funcional con pruebas automatizadas.
4. Pipeline de integración continua.
5. Construcción y publicación de una imagen de contenedor.
6. Despliegue automatizado.
7. Infraestructura o configuración definida como código.
8. Gestión de variables y secretos.
9. Monitoreo de la aplicación.
10. Recolección de métricas y registros.
11. Configuración de al menos una alerta.
12. Procedimiento de reversión o recuperación.
13. Documentación técnica.
14. Presentación y demostración final.

13. Evaluación y condiciones de aprobación

13.1. Evaluación propuesta

Instancia	Porcentaje
Trabajos prácticos y laboratorios	30 %
Evaluaciones parciales	20 %
Proyecto integrador	40 %
Presentación y defensa final	10 %

13.2. Criterios de evaluación

- Comprensión de los conceptos.
- Correcta implementación técnica.
- Nivel de automatización alcanzado.
- Reproducibilidad de la solución.
- Calidad del repositorio.
- Seguridad de las configuraciones.
- Capacidad para diagnosticar errores.
- Calidad de la documentación.
- Trabajo colaborativo.
- Presentación y defensa de las decisiones adoptadas.

13.3. Requisitos de regularidad

- Un mínimo del 75 % de asistencia.
- Aprobación de al menos el 80 % de los trabajos prácticos.
- Aprobación de las evaluaciones parciales o sus recuperatorios.
- Presentación obligatoria del proyecto integrador.
- Calificación final mínima de 6 sobre 10, o la establecida por la institución.

13.4. Recuperación

Los estudiantes podrán recuperar las evaluaciones parciales no aprobadas y corregir los trabajos prácticos observados dentro de los plazos establecidos.

El proyecto integrador podrá contar con una instancia de revisión previa a la presentación definitiva.

14. Cronograma propuesto

Semana	Contenido principal	Actividad
1	Introducción a DevOps	Análisis del ciclo de vida
2	Linux y línea de comandos	Laboratorio Linux
3	Redes y scripting	Script de automatización
4	Git y trabajo colaborativo	Repositorio grupal
5	Integración continua	Pipeline inicial
6	Pruebas y artefactos	Automatización de pruebas
7	Contenedores	Construcción de imágenes
8	Aplicaciones multicontenedor	Servicios integrados
9	Orquestación	Despliegue en Kubernetes
10	Entrega continua	Estrategias de despliegue
11	Infraestructura como código	Aprovisionamiento
12	Monitoreo y métricas	Tablero de observabilidad
13	Registros, alertas e incidentes	Simulación de falla
14	DevSecOps	Seguridad del pipeline
15	Integración del proyecto	Pruebas finales
16	Presentación y defensa	Demostración del proyecto

15. Recursos tecnológicos sugeridos

El desarrollo de la asignatura podrá utilizar:

- Linux.
- Git.
- GitHub, GitLab o plataforma equivalente.
- Bash o Python.
- Docker o tecnología compatible.
- Kubernetes.
- Herramientas de integración continua.
- Herramientas de infraestructura como código.
- Sistemas de monitoreo y visualización.
- Herramientas de recolección de registros.
- Plataformas de nube pública, privada o entornos locales.

La selección de herramientas deberá considerarse instrumental. Los objetivos del programa deben mantenerse aun cuando las tecnologías utilizadas sean reemplazadas o actualizadas.

16. Bibliografía

16.1. Bibliografía básica

1. KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. The DevOps Handbook.
2. FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps.
3. HUMBLE, Jez; FARLEY, David. Continuous Delivery.
4. BEYER, Betsy; JONES, Chris; PETOFF, Jennifer; MURPHY, Niall Richard. Site Reliability Engineering.
5. MORRIS, Kief. Infrastructure as Code.
6. BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey. Kubernetes: Up and Running.
7. TURNAULL, James. The Docker Book.
8. KIM, Gene; BEHR, Kevin; SPAFFORD, George. The Phoenix Project.

16.2. Bibliografía complementaria

- Documentación oficial de Git.
- Documentación oficial de Docker.
- Documentación oficial de Kubernetes.
- Documentación oficial de herramientas de integración continua.
- Documentación oficial de plataformas de infraestructura como código.
- Documentación oficial de Prometheus, Grafana y OpenTelemetry.
- Documentación de proveedores de servicios en la nube.
- Guías de seguridad para desarrollo y despliegue de software.

17. Vigencia y revisión del programa

Debido a la evolución permanente de las tecnologías utilizadas en DevOps, el programa deberá revisarse periódicamente.

La revisión deberá considerar:

- Actualización de herramientas.
- Cambios en las prácticas de seguridad.
- Nuevas formas de automatización.
- Evolución de las plataformas de contenedores.
- Incorporación de nuevas prácticas de observabilidad.
- Necesidades del mercado profesional.
- Resultados obtenidos por los estudiantes.
- Recomendaciones del equipo docente.

Se recomienda realizar una revisión menor anual y una revisión integral cada dos o tres años.

Profesor/a responsable

Coordinación académica

Autoridad institucional